

FILE HANDLING IN PYTHON

Beginner Level – Professional Notes

1 . Introduction

File handling in Python allows us to work with external files to store and retrieve data. It is in real-world applications for saving records, reports, logs, and structured data.

Using file handling, we can: - Create a file - Write data into a file - Read data from a file
Update existing data - Delete a file

Python commonly works with three file types: - TXT (Text Files) - CSV (Comma Separated Values) (JavaScript Object Notation)

2 . TXT File Handling

TXT files are used to store plain text data.

2 . 1 Create / Write a TXT File ("w" mode)

```
with open("sample.txt", "w") as file:  
    file.write("Hello Students")
```

- Creates the file if it does not exist.
 - Overwrites the file if it already exists.
-

2 . 2 Read a TXT File ("r" mode)

```
with open("sample.txt", "r") as file:  
    content = file.read()  
    print(content)
```

2 . 3 Append to a TXT File ("a" mode)

```
with open("sample.txt", "a") as file:  
    file.write("\nWelcome to Python")
```

- Adds new content at the end of the file.
 - Does not remove existing content.
-

2 . 4 Update a TXT File

```
with open("sample.txt", "r") as file:  
    data = file.read()  
  
updated_data = data.replace("Hello", "Hi")  
  
with open("sample.txt", "w") as file:  
    file.write(updated_data)
```

Process: Read → Modify → Write again.

2 . 5 Delete a TXT File

```
import os  
os.remove("sample.txt")
```

3 . CSV File Handling

CSV files are used to store tabular data (rows and columns), similar to Excel.

3 . 1 Import CSV Module

```
import csv
```

3 . 2 Create / Write a CSV File

```
with open("students.csv", "w", newline="") as file:
    writer = csv.writer(file)
    writer.writerow(["Name", "Age"])
    writer.writerow(["Maha", 25])
```

- Each row is written using `writerow()`.
 - `newline=""` prevents extra blank lines.
-

3 . 3 Read a CSV File

```
with open("students.csv", "r") as file:
    reader = csv.reader(file)
    for row in reader:
        print(row)
```

- Each row is returned as a list.
-

3 . 4 Append Data to CSV

```
with open("students.csv", "a", newline="") as file:
    writer = csv.writer(file)
    writer.writerow(["Arun", 22])
```

3 . 5 Update a CSV File

```
rows = []

with open("students.csv", "r") as file:
    reader = csv.reader(file)
    rows = list(reader)

rows[1][1] = "26"    # Modify specific value

with open("students.csv", "w", newline="") as file:
```

```
writer = csv.writer(file)
writer.writerows(rows)
```

Process: Read → Modify → Write again.

3 . 6 Delete a CSV File

```
import os
os.remove("students.csv")
```

4 . JSON File Handling

JSON files are used to store structured data in key-value (dictionary) format.

4 . 1 Import JSON Module

```
import json
```

4 . 2 Create / Write a JSON File

```
data = {"name": "Maha", "course": "Excel"}

with open("data.json", "w") as file:
    json.dump(data, file)
```

4 . 3 Read a JSON File

```
with open("data.json", "r") as file:
    data = json.load(file)
    print(data)
```

4 . 4 Update a JSON File

```
with open("data.json", "r") as file:
    data = json.load(file)

data["fee"] = 199 # Add or update key

with open("data.json", "w") as file:
    json.dump(data, file)
```

Process: Read → Modify → Write again.

4 . 5 Delete a JSON File

```
import os
os.remove("data.json")
```

5 . File Modes Summary

Mode	Meaning
r	Read
w	Write (overwrite)
a	Append
r+	Read and Write

6 . Important Points

- Always use "with open()" as it automatically closes the file.
 - "w" mode deletes existing content before writing.
 - "a" mode adds content without deleting existing data.
 - CSV files are suitable for table data.
 - JSON files are suitable for structured dictionary data.
 - Updating a file generally follows: Read → Modify → Write.
-

End of Document